

Exam “Algorithms” WS 2021/2022

February 24, 2022

Exam duration: 90 minutes

Remarks

- (1) Use a blue or black ballpoint pen, or a fountain pen. (Neither pencils nor red pens.)
- (2) Calculators, lecture notes, cheat sheets, and other *material* are *not* allowed. Mobile phones and similar devices like smart watches must be turned off and packed away.
- (3) You can work on the tasks directly on the work sheets. *Additional sheets* will be handed out at request. Write your name and student ID number on these, too.
- (4) Some tasks contain texts with gaps “.....”. Here you are supposed to fill in a formula, one word or several words, or a short sentence (even if there is no explicit instruction).
- (5) Your answers, e. g., bounds in big-O notation, should be meaningful: “An array of size n can be sorted in time $O(\dots)$ ” is not wrong, but not worth a point.

Task	1	2	3	4	5	6	7	8	9	Bonus	Total	Grade
Max. score	18	9	9	5	8	10	13	10	8	5	90 + 5	
Achieved score	...	...	...	...	...	...	...	...	...	...	...	...

Fill in at the beginning of the exam:

 Last name, first name

 Student ID number

 Degree program
 (probably: RCSE)

In addition, _____ sheets were handed in.

Fill in at post-exam review:

 Date, signature

Task 1 (Mixed Topics)

[18 points]

Fill in the gaps. If not specified in a different way, each correct entry gives 1 point, and each wrong or omitted entry gives 0 points.

(a) Big-O notation

(4 p.)

In this subtask, each correct entry gives 1/2 point. Let $f: \mathbb{N} \rightarrow \mathbb{R}^+$ and $g: \mathbb{N} \rightarrow \mathbb{R}^+$ be functions.

- We say " $g(n) = O(f(n))$ " if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

..... for all

- *Domination rule:* If $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$, then = $O(f(n))$.

- If $f(n) = \frac{(n^4 + n^3 \log(n)) \cdot 8^{n/3}}{6n + 3n\sqrt{n}}$, then $f(n) = \Theta(n^a \cdot b^n)$ for constants $a = \dots$ and $b = \dots$.

- Let $n \in \mathbb{N}$. Then $\sum_{i=1}^n \frac{1}{i} = \Theta(\dots)$.

- If $f(n) = n \sin^2(n) + 2n \cos^2(n)$, then $f(n) = O(\dots)$ and $f(n) = \Omega(\dots)$.

(b) Modular arithmetic

(5 p.)

Let $x, y, N \in \mathbb{N}$ be n -bit integers such that $N \geq 2$.

- If $x < N$, the value $x^y \bmod N$ can be computed using the following recursive formula:

$$x^y \bmod N = \begin{cases} 1 & , \text{ if } y = 0 \\ x & , \text{ if } y = 1 \\ \dots & , \text{ if } y \geq 2 \text{ even} \\ \dots & , \text{ if } y \geq 2 \text{ odd} \end{cases}$$

The number of bit operations when using this method is $O(\dots)$.

- If $x \geq y$, the value $\gcd(x, y)$ can be computed using the following recursive formula:

$$\gcd(x, y) = \begin{cases} x & , \text{ if } y = 0 \\ \dots & , \text{ if } y \geq 1 \end{cases}$$

The number of bit operations when using this method is $O(\dots)$.

Continued on next page!

(c) Divide-and-Conquer algorithms

(5 p.)

- Karatsuba's algorithm multiplies two n -bit integers in time $O(\dots\dots)$.

When the algorithm is carried out, it calls itself $\dots\dots\dots$ times at the first level of recursion.

- Randomized Quickselect finds the element of rank k of n elements in expected time $O(\dots\dots)$.
- Randomized Quicksort sorts n distinct elements in expected time $O(\dots\dots\dots)$.
- If $\mathcal{A}$ is a comparison-based sorting algorithm, it requires time $\Omega(\dots\dots\dots)$ (worst case) to sort n elements.

(d) Edit distance

(4 p.)

Calculate the edit distance of the strings $x = \text{FLOSS}$ and $y = \text{CLUES}$ by filling in the following matrix.

E		C	L	U	E	S
	0	1	2	3	4	5
F	1					
L	2					
O	3					
S	4					
S	5					

The edit distance of FLOSS and CLUES is $\dots\dots\dots$.

Task 2 (Primality Testing with the Fermat Test)

[9 points]

(a) Formulate the Fermat test:

(4 p.)

Input: Odd integer $N \geq 3$ **Method:**

```

1  choose integer  $a$  ..... from .....;
2  calculate integer  $b \leftarrow$  .....;
3  if ..... then return 1 else return 0;

```

(b) Describe the **output behavior** of the Fermat test:

(2 p.)

- If N is prime, then the output is always
- If N is a composite non-Carmichael number, then $\Pr(\text{output is 0})$
(E. g. " $= \frac{1}{3}$ " or " $\geq \frac{5}{7}$ " or " < 1 " or ...)

(c) Let $N \geq 9$ be some composite odd number. N is called a **Carmichael number** if

(2 p.)

.....

.....

(d) The expected running time for generating a random n -bit prime with error probability at most $4^{-\ell}$ is $O(\dots)$.

(1 p.)

Task 3 (RSA)

[9 points]

We consider the RSA cryptosystem as discussed in the videos and the notes. In the following three parts, fill in the missing words or formulas.

(a) Preprocessing / Key Generation

(6 p.)

We choose two distinct primes p and q , define $N = \dots\dots\dots$, and calculate $\varphi(N) = \dots\dots\dots$. For Bob's public key (N, e) , we choose some integer e such that $3 \leq e < \varphi(N)$ and Bob's private key is (N, d) , where integer d , $3 \leq d < \varphi(N)$, is such that d can be calculated using the algorithm.

(b) Communication

(1 p.)

Alice has a message/plaintext $x \in [N] = \{0, 1, \dots, N-1\}$. She computes $y = \dots\dots\dots$ and sends y to Bob. Bob receives y and computes $z = \dots\dots\dots$.

(c) Correctness Claim / Decryption Condition

(2 p.)

..... for all $x \in [N]$.

Task 4 (Proofs)

[5 points]

Prove **one** of the following statements. 3 You may also prove the other one for 5 **bonus** points.

- (a) Let $p \geq 3$ be a prime number and consider the set $S = \{1, \dots, p-1\}$. For any natural number $1 \leq a < p$, the size of the set $S_a = \{(a \cdot x) \bmod p \mid x \in S\}$ is $|S_a| = p-1$.

.....

.....

.....

.....

.....

.....

.....

.....

- (b) In the setting of task 3 (RSA), $(x^{ed} - x) \bmod p = 0$ holds for all $x \in [N]$ where $p \nmid x$.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

*The first statement is used in the proof of Fermat's Little Theorem. The second one is important for the decryption condition.

Task 5 (Recurrence Relations)

[8 points]

- (a) Consider some algorithm A . It solves instances of size $n = 1$ in constant time. If $n > 1$, it divides the input into 4 sub-instances of size $\lfloor n/2 \rfloor$ in time $O(n^2)$, solves these recursively, and combines the solutions to obtain a solution for the original instance in time $O(n^2 \log n)$.

Let $T(n)$ be the running time of A on inputs of size $n \geq 1$. It fulfills the following recurrence relation:

$$T(n) \leq \begin{cases} \dots\dots\dots, & \text{if } n = 1 \\ \dots\dots\dots, & \text{if } n > 1 \end{cases} \quad (2 \text{ p.})$$

- (b) Consider some algorithm B . It solves problems of size $n \geq 1$ in time $T(n)$, where

$$T(n) \leq \begin{cases} 4, & \text{if } n = 1 \\ 8 \cdot T\left(\frac{n}{3}\right) + 5 \cdot n^3 + n \log n, & \text{if } n > 1 \end{cases}$$

Solve this recurrence relation using the **Master theorem**. What are the values of the important constants a , b , and d ? Which case of the Master theorem applies? (3 p.)

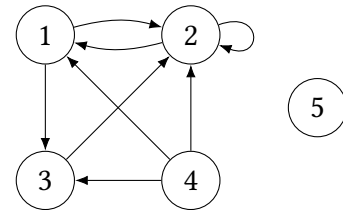
- (c) Consider some algorithm C . On input $n \geq 1$ it calculates $f(n)$, where

$$f(n) = \begin{cases} 1, & \text{if } n = 1 \\ 5 \cdot f(n-1) - 1, & \text{if } n > 1 \end{cases}$$

Solve this recurrence relation **exactly without using big-O notation**. (You will need a direct calculation. The Master theorem does not help here.) (3 p.)

Task 6 (Digraph Representations)

[10 points]

(a) Consider the directed graph G as depicted on the right.Write down G using the following representations:

(6 p.)

(i) Adjacency matrix

	1	2	3	4	5
1					
2					
3					
4					
5					

(ii) Adjacency lists (Use $\boxed{a \bullet}$ for list elements with entry a and non-null pointers, and $\boxed{/}$ for null pointers.)

1	
2	
3	
4	
5	

(iii) Adjacency array

	1	2	3	4	5	6	7	8
neighbor:								
start:								
	1	2	3	4	5	6		

(b) Let $G = (V, E)$ be a digraph with $|V| = n$ and $|E| = m$. For a node $v \in V$, $\text{outdeg}(v)$ is the number of outgoing edges $(v, w) \in E$ and $\text{indeg}(v)$ is the number of ingoing edges $(u, v) \in E$. What is the **time** needed to compute these numbers if G is given in one of the following representations?[†] (4 p.)

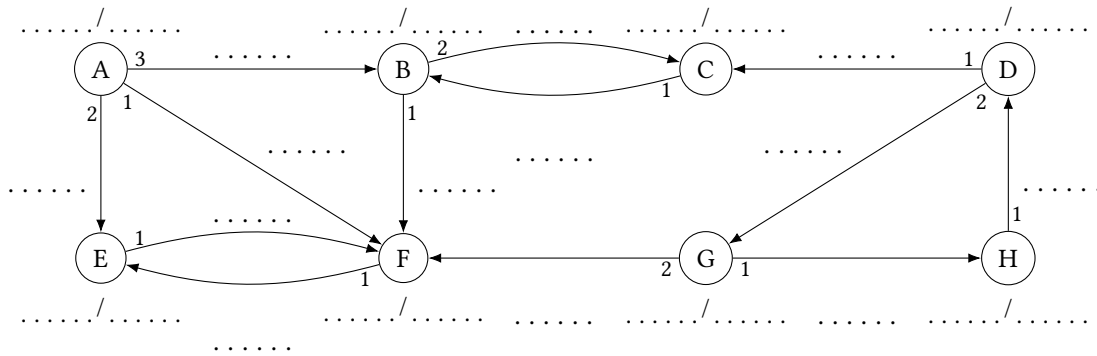
	Adjacency lists	Adjacency array
$\text{outdeg}(v)$:	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$
$\text{indeg}(v)$:	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$

[†]In general, $O(n + m)$ is not the same as $O(n)$ or $O(m)$. A $O(\text{outdeg}(v))$ bound can be much smaller than $O(n)$ and $O(m)$.

Task 7 (Depth-First Search – DFS)

[13 points]

- (a) Perform **depth-first search** on the following digraph. The nodes are ordered alphabetically, i. e., start with node A. The numbers on outgoing edges of a node indicate the ordering of its neighbors in its adjacency list. (5 p.)



- (b) What are the **strongly connected components** of the digraph in (a)? Give the node sets! (1 p.)

.....

How does **Kosaraju's algorithm** compute strongly connected components of a digraph G ? (3 p.)

(1) Run DFS on

(2) Calculate

(3) Run DFS on where in the outer loop

.....

.....

(4) Output: node sets of

What is the running time of Kosaraju's algorithm on input $G = (V, E)$, where $|V| = n$ and $|E| = m$? (1 p.)

$O(\dots\dots\dots)$

- (c) A **topological ordering** of a digraph $G = (V, E)$ is an ordering $(v_1, \dots, v_n)$ of the nodes such that (1 p.)

.....

Consider the digraph G and the DFS run from (a). Let G' be the digraph obtained by removing all the B-edges from G . Observe that G' is a DAG and a DFS run on G' yields the same pre and post numbers. Give an explicit topological ordering of G' . (2 p.)

.....

Task 8 (Minimum Spanning Trees – Jarník/Prim and Kruskal's Algorithm)

[10 points]

In this task, $G = (V, E, c)$ is a connected (undirected) graph with edge weights $c(e) \in \mathbb{R}$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

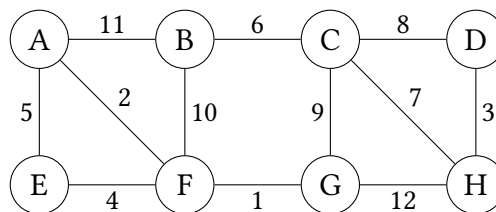
(a) Let $T \subseteq E$ be a set of edges in G . (3 p.)

(i) T is a **spanning tree** for G if

(ii) The **weight** of T is $c(T) :=$

(iii) T is a **minimum spanning tree** for G if

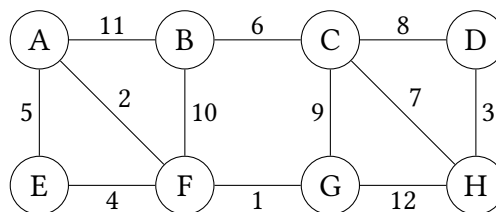
(b) Run the **Jarník/Prim algorithm** on the following undirected weighted graph with start node A by listing all edges of the resulting minimum spanning tree T in the order in which they are chosen by the algorithm. Calculate the weight of T .



Chosen edges: (A, F), (2 p.)

$c(T) =$ (1 p.)

(c) Run **Kruskal's algorithm** on the following undirected weighted graph by listing all edges of the resulting minimum spanning tree T in the order in which they are chosen by the algorithm.



Chosen edges: (2 p.)

(d) The Jarník/Prim algorithm finds a minimum spanning tree in time $O(\dots)$ using a priority queue, realized as a binary heap. (1 p.)

Kruskal's algorithm finds a minimum spanning tree in time $O(\dots)$ using the union-find data structure, realized as trees with ranks. (1 p.)

Task 9 (Optimal Binary Coding Trees – Huffman's Algorithm)

[8 points]

- (a) Consider the coding tree T depicted on the right with the following relative frequencies: $p(A) = 0.4, p(M) = 0.3, p(R) = 0.2, p(T) = 0.1$.

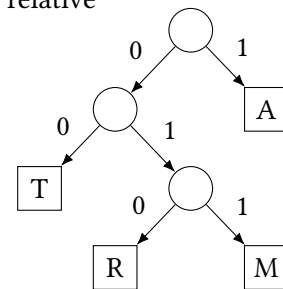
(2 p.)

- (i) Decode 00101110101.

.....

- (ii) Calculate the cost of T .

$$B(T, p) =$$



Hint: State the concrete number, not (only) the formula.

- (b) The following table gives the frequencies of the letters A, B, ..., G in an unknown language. (The entries have already been sorted according to frequencies.) Run **Huffman's algorithm** on this input by filling in the table. (4 p.)

a_i	F	B	G	C	D	A	E						
i	1	2	3	4	5	6	7	8	9	10	11	12	13
p_i	0.03	0.04	0.07	0.10	0.10	0.19	0.47	0.07	0.14	0.20	0.33	0.53	1
pred	8	8	9	10	10	11	13	9	11	12	12	13	–
mark	0	1	0	0	1	1	0	1	0	0	1	1	–

- (c) Draw the resulting coding tree.

(2 p.)

All the best!

Exam “Algorithms” SS 2021

September 21, 2021

Exam duration: 90 minutes

Remarks

- (1) Use a blue or black ballpoint pen, or a fountain pen. (Neither pencils nor red pens.)
- (2) Calculators, lecture notes, cheat sheets, and other *material* are *not* allowed. Mobile phones and similar devices like smart watches must be turned off and packed away.
- (3) You can work on the tasks directly on the work sheets. *Additional sheets* will be handed out at request. Write your name and student ID number on these, too.
- (4) Some tasks contain texts with gaps “.....”. Here you are supposed to fill in a formula, one word or several words, or a short sentence (even if there is no explicit instruction).
- (5) Your answers, e. g., bounds in big-O notation, should be meaningful: “An array of size n can be sorted in time $O(n^{42})$ ” is not wrong, but not worth a point.

Task	1	2	3	4	5	6	7	8	9	Total	Grade
Max. score	16	8	7	7	14	11	9	10	8	90	
Achieved score	...	...	...	...	...	...	...	...	...	...	...

Fill in at the beginning of the exam:

 Last name, first name

 Student ID number

 Degree program
 (probably: RCSE)

In addition, _____ sheets were handed in.

Fill in at post-exam review:

 Date, signature

Task 1 (Mixed Topics)

[16 points]

Fill in the gaps. If not specified in a different way, each correct entry gives 1 point, and each wrong or omitted entry gives 0 points.

(a) Big-O notation

(5 p.)

In this subtask, each correct entry gives 1/2 point. Let $f: \mathbb{N} \rightarrow \mathbb{R}^+$ and $g: \mathbb{N} \rightarrow \mathbb{R}^+$ be functions.

- We say " $g(n) = \Omega(f(n))$ " if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

..... for all

- *Limit rule:* If $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = c$ for some $c \geq 0$, then $g(n) = \dots\dots\dots$.

Domination rule: If $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$, then = $O(f(n))$.

- If $f(n) = \frac{(n^4 + n^2) \cdot 8^{n/3}}{7n^5 - 4n\sqrt{n}}$, then $f(n) = \Theta(n^a \cdot b^n)$ for constants $a = \dots\dots\dots$ and $b = \dots\dots\dots$.
- Let $n \in \mathbb{N}$. Then $\sum_{i=0}^n i = \Theta(\dots\dots\dots)$ and $\sum_{i=1}^n \frac{1}{i} = \Theta(\dots\dots\dots)$.
- If $f(n) = 7 - 3 \sin(n)$, then $f(n) = O(\dots\dots\dots)$ and $f(n) = \Omega(\dots\dots\dots)$.

(b) Modular arithmetic

(6 p.)

Let $x, y, N \in \mathbb{N}$ be n -bit integers such that $N \geq 2$.

- The value $(x \cdot y) \bmod N$ can be computed naively using grade school methods.

The number of bit operations when using these methods is $O(\dots\dots\dots)$.

- If $x < N$, the value $x^y \bmod N$ can be computed using the following recursive formula:

$$x^y \bmod N = \begin{cases} 1 & , \text{ if } y = 0 \\ x & , \text{ if } y = 1 \\ \dots\dots\dots & , \text{ if } y \geq 2 \text{ even} \\ \dots\dots\dots & , \text{ if } y \geq 2 \text{ odd} \end{cases}$$

The number of bit operations when using this method is $O(\dots\dots\dots)$.

- If $x \geq y$, the value $\gcd(x, y)$ can be computed using the following recursive formula:

$$\gcd(x, y) = \begin{cases} x & , \text{ if } y = 0 \\ \dots\dots\dots & , \text{ if } y \geq 1 \end{cases}$$

The number of bit operations when using this method is $O(\dots\dots\dots)$.

Continued on next page!

(c) Divide-and-Conquer algorithms

(5 p.)

- Karatsuba's algorithm multiplies two n -bit integers in time $O(\dots\dots\dots)$.

When the algorithm is carried out, it is called recursively $\dots\dots\dots$ times (on the first level only).

- Randomized Quickselect finds the element of rank k of n elements in expected time $O(\dots\dots\dots)$.
- Randomized Quicksort sorts n distinct elements in expected time $O(\dots\dots\dots)$.
- If $\mathcal{A}$ is a comparison-based sorting algorithm, it requires time $\Omega(\dots\dots\dots)$ (worst case) to sort n elements.

Task 2 (Primality Testing with the Fermat Test)

[8 points]

(a) Formulate the Fermat test:

(4 p.)

Input: Odd integer $N \geq 3$ **Method:**

```

1 | choose integer  $a$  ..... from .....;
2 | calculate integer  $b \leftarrow$  .....;
3 | if ..... then return 1 else return 0;

```

(b) Describe the **output behavior** of the Fermat test:

(2 p.)

- If N is prime, then the output is always
- If N is a composite non-Carmichael number, then $\Pr(\text{output is 0})$
(E. g. " $= \frac{1}{3}$ " or " $\geq \frac{5}{7}$ " or " < 1 " or ...)

(c) Let $N \geq 9$ be some composite odd number. N is called a **Carmichael number** if

(1 p.)

.....

.....

(d) The expected running time for generating a random n -bit prime with error probability $\leq 4^{-\ell}$ is (1 p.)

$O(\text{.....})$.

Task 3 (RSA)

[7 points]

We consider the RSA cryptosystem as discussed in the videos and the notes. In the following three parts, fill in the missing words or formulas.

(a) Preprocessing/Key generation

(5 p.)

We randomly choose two distinct primes p and q , define $N = \dots$, and calculate $\varphi(N) = \dots$. For Bob's public key (N, e) , we choose some integer e , $3 \leq e < \varphi(N)$, such that $\dots$. Bob's private key is (N, d) , where integer d , $3 \leq d < \varphi(N)$, is such that $\dots$. d can be calculated using the $\dots$ algorithm.

(b) Communication

(1 p.)

Alice has a message/plaintext $x \in [N] = \{0, 1, \dots, N-1\}$. She computes $y = \dots$ and sends y to Bob. Bob receives y and computes $z = \dots$.

(c) Correctness claim/Decryption condition

(1 p.)

$\dots \bmod \dots = x$ for all $x \in [N]$.

Task 4 (Recurrence Relations)

[7 points]

- (a) Consider some algorithm A . It solves instances of size $n = 1$ in constant time. If $n > 1$, it divides the input into 4 sub-instances of size $\lfloor n/3 \rfloor$ in time $O(n^2)$, solves these recursively, and combines the solutions to obtain a solution of the original input in time $O(n(\log n)^3)$.

Let $T(n)$ be the running time of A on inputs of size $n \geq 1$. It fulfills the following recurrence relation:

$$T(n) \leq \begin{cases} \dots\dots\dots, & \text{if } n = 1 \\ \dots\dots\dots, & \text{if } n > 1 \end{cases} \quad (2 \text{ p.})$$

- (b) Consider some algorithm B . It solves problems of size $n \geq 1$ in time $T(n)$, where

$$T(n) \leq \begin{cases} 4, & \text{if } n = 1 \\ 4 \cdot T(\frac{n}{2}) + 4 \cdot n^2 - n \log n, & \text{if } n > 1 \end{cases}$$

Solve this recurrence relation using the **Master theorem**. What are the values of the important constants a , b , and d ? Which case of the Master theorem applies? (3 p.)

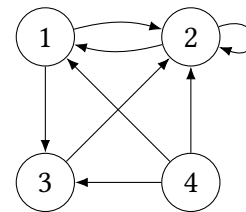
- (c) Consider some algorithm C . On input $n \geq 1$ it calculates $f(n)$, where

$$f(n) = \begin{cases} 1, & \text{if } n = 1 \\ 3 \cdot f(n-1) + 1, & \text{if } n > 1 \end{cases}$$

Solve this recurrence relation **exactly without big-O notation**. (You will need a direct calculation. The Master theorem does not help here.) (2 p.)

Task 5 (Digraph Representations)

[14 points]

(a) Consider the directed graph G as depicted on the right.Write down G using the following representations:

(6 p.)

(i) Adjacency matrix

	1	2	3	4	5
1					
2					
3					
4					
5					

(ii) Adjacency lists (Use $\boxed{a} \bullet \rightarrow$ for list elements with entry a and non-null pointers, and $\boxed{/}$ for null pointers.)

1	
2	
3	
4	
5	

(iii) Adjacency array

	1	2	3	4	5	6	7	8
neighbor:								
start:								
	1	2	3	4	5	6		

(b) Draw the **reverse graph** of the digraph depicted in (a).

(2 p.)

(c) Let $G = (V, E)$ be a digraph with $|V| = n$ and $|E| = m$. For a node $v \in V$, $\text{outdeg}(v)$ is the number of outgoing edges $(v, w) \in E$ and $\text{indeg}(v)$ is the number of ingoing edges $(u, v) \in E$. What is the **time** needed to compute these numbers if G is given in one of the following representations?¹

(6 p.)

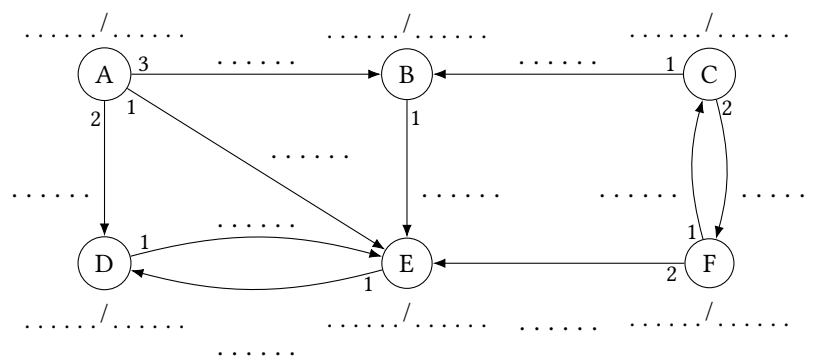
	Adjacency matrix	Adjacency lists	Adjacency array
$\text{outdeg}(v)$:	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$
$\text{indeg}(v)$:	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$

¹In general, $O(n + m)$ is not the same as $O(n)$ or $O(m)$. A bound $O(\text{outdeg}(v))$ can be much smaller than $O(n)$ and $O(m)$.

Task 6 (Depth-First Search – DFS)

[11 points]

- (a) Let $G = (V, E)$ be a digraph with $n = |V|$ nodes and $m = |E|$ edges. **DFS** assigns each node two numbers called, and each edge is labeled with one of the following edge labels: The running time is $O(\dots\dots\dots)$.
- (b) Perform **depth-first search** on the following digraph. The nodes are ordered alphabetically, i. e., start with node A. The numbers on outgoing edges of a node indicate the ordering of its neighbors in its adjacency list. (6 p.)



- (c) A **topological ordering** of a digraph $G = (V, E)$ is an ordering $(v_1, \dots, v_n)$ of the nodes such that (2 p.)

.....
 Given the result of a DFS run in G , a topological ordering of G can be computed as follows (if it exists):

.....

Task 7 (Shortest Paths – Dijkstra's Algorithm)

[9 points]

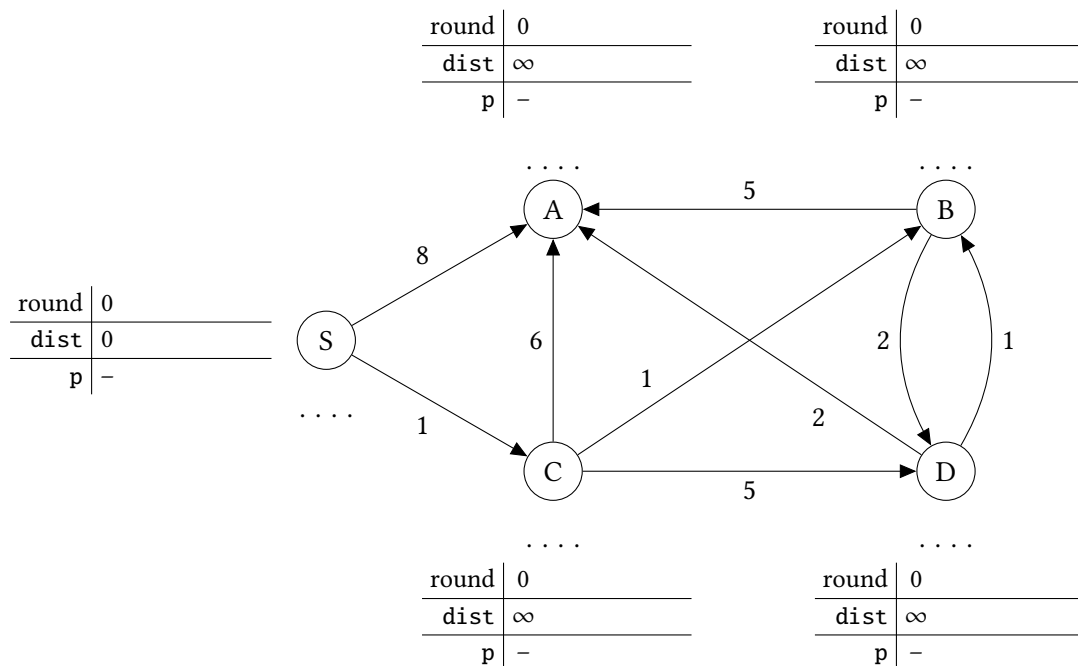
(a) Dijkstra's algorithm can be applied when a weighted digraph $G = (V, E, c)$ and a start node $s \in V$ are given. It calculates the distances $d(s, v)$, for all $v \in V$. All edge weights $c(u, v)$, $(u, v) \in E$, must be The running time of Dijkstra's algorithm (using a priority queue, implemented as a binary heap) is $O(\dots)$, where $n = |V|$ and $m = |E|$. (2 p.)

(b) Run Dijkstra's algorithm on the following weighted digraph, starting at node S, by filling in the gaps and tables as follows: (5 p.)

- If node v is **scanned** in round i , fill in the dotted gap next to v with i .
- If node v is **found** or its distance value $\text{dist}[v]$ is **changed (decreased)** in round i , add a new column to v 's table recording the round number i , the new distance value $\text{dist}[v]$, and the corresponding predecessor $p[v]$.

Ⓢ **13**

round	0	2	5	11
dist	∞	24	12	10
p	-	X	Y	Z



Draw the resulting shortest-path tree.

(2 p.)

Task 8 (Minimum Spanning Trees – Jarník/Prim and Kruskal's Algorithm)

[10 points]

In this task, $G = (V, E, c)$ is a connected (undirected) graph with edge weights $c(e) \in \mathbb{R}$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

(a) Let $T \subseteq E$ be a set of edges in G . (3 p.)

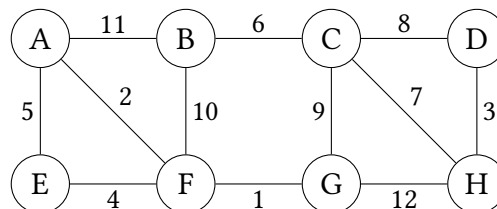
(i) T is a **spanning tree** for G if

(ii) The **weight** of T is $c(T) :=$

(iii) T is a **minimum spanning tree** for G if

.....

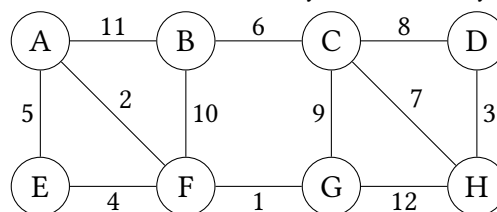
(b) Run the **Jarník/Prim algorithm** on the following undirected weighted graph with start node A by listing all edges of the resulting minimum spanning tree T in the order in which they are chosen by the algorithm. Calculate the weight of T .



Chosen edges: (A, F), (2 p.)

$c(T) =$ (1 p.)

(c) Run **Kruskal's algorithm** on the following undirected weighted graph by listing all edges of the resulting minimum spanning tree T in the order in which they are chosen by the algorithm.



Chosen edges: (2 p.)

(d) The Jarník/Prim algorithm finds a minimum spanning tree in time $O(\dots)$ using a priority queue, realized as a binary heap. (1 p.)

Kruskal's algorithm finds a minimum spanning tree in time $O(\dots)$ using the union-find data structure, realized as trees. (1 p.)

Task 9 (Optimal Binary Coding Trees – Huffman's Algorithm)

[8 points]

- (a) Consider the coding tree T depicted on the right with the following relative frequencies: $p(A) = 0.4, p(M) = 0.3, p(R) = 0.2, p(T) = 0.1$.

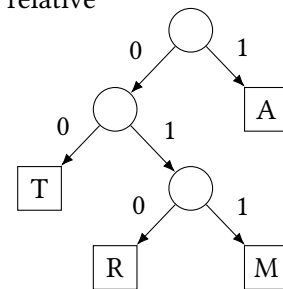
(2 p.)

- (i) Decode 10101010100.

.....

- (ii) Calculate the cost of T .

$$B(T, p) =$$



- (b) The following table gives the frequencies of the letters A, B, ..., G in an unknown language. (The entries have already been sorted according to frequencies.) Run **Huffman's algorithm** on this input by filling in the table. (4 p.)

a_i	F	B	G	C	D	A	E						
i	1	2	3	4	5	6	7	8	9	10	11	12	13
p_i	0.03	0.03	0.06	0.11	0.11	0.21	0.45						
pred													–
mark													–

Draw the corresponding optimal coding tree.

(2 p.)

Good luck!

Exam “Algorithms” WS 2020/2021

March 26, 2021

Exam duration: 90 minutes

Remarks

- (1) Use a blue or black ballpoint pen, or a fountain pen. (Neither pencils nor red pens.)
- (2) Calculators, lecture notes, cheat sheets, and other *material* are *not* allowed. Mobile phones and similar devices like smart watches must be turned off and packed away.
- (3) You can work on the tasks directly on the work sheets. *Additional sheets* will be handed out at request. Write your name and student ID number on these, too.
- (4) Some tasks contain texts with gaps “.....”. Here you are supposed to fill in a formula, one word or several words, or a short sentence (even if there is no explicit instruction).
- (5) Your answers, e. g., bounds in big-O notation, should be meaningful: “An array of size n can be sorted in time $O(n^{42})$ ” is not wrong, but not worth a point.

Task	1	2	3	4	5	6	7	8	9	Total	Grade
Max. score	16	8	6	7	14	11	9	11	8	90	
Achieved score	...	...	...	...	...	...	...	...	...	...	...

Fill in at the beginning of the exam:

 Last name, first name

 Student ID number

 Degree program
 (probably: RCSE)

In addition, _____ sheets were handed in.

Fill in at post-exam review:

 Date, signature

Task 1 (Mixed Topics)

[16 points]

Fill in the gaps. If not specified in a different way, each correct entry gives 1 point, and each wrong or omitted entry gives 0 points.

(a) Big-O notation

(5 p.)

In this subtask, each correct entry gives 1/2 point. Let $f: \mathbb{N} \rightarrow \mathbb{R}^+$ and $g: \mathbb{N} \rightarrow \mathbb{R}^+$ be functions.

- We say " $g(n) = \Omega(f(n))$ " if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

..... for all

- *Limit rule:* If $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = c$ for some $c \geq 0$, then

Domination rule: If $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$, then

- If $f(n) = \frac{(n^3 - n^2) \cdot 8^{n/3}}{7n^4 + 5n\sqrt{n}}$, then $f(n) = \Theta(n^a \cdot b^n)$ for constants $a = \dots$ and $b = \dots$.

- Let $n \in \mathbb{N}$. Then $\sum_{i=0}^n i = \Theta(\dots)$ and $\sum_{i=1}^n \frac{1}{i} = \Theta(\dots)$.

- If $f(n) = 5 - 2 \sin(n)$, then $f(n) = O(\dots)$ and $f(n) = \Omega(\dots)$.

(b) Modular arithmetic

(6 p.)

Let $x, y, N \in \mathbb{N}$ be n -bit integers such that $N \geq 2$.

- The value $(x \cdot y) \bmod N$ can be computed naively using grade school methods.

The number of bit operations when using these methods is $O(\dots)$.

- If $x < N$, the value $x^y \bmod N$ can be computed using the following recursive formula:

$$x^y \bmod N = \begin{cases} 1 & , \text{ if } y = 0 \\ x & , \text{ if } y = 1 \\ \dots & , \text{ if } y \geq 2 \text{ even} \\ \dots & , \text{ if } y \geq 2 \text{ odd} \end{cases}$$

The number of bit operations when using this method is $O(\dots)$.

- If $x \geq y$, the value $\gcd(x, y)$ can be computed using the following recursive formula:

$$\gcd(x, y) = \begin{cases} x & , \text{ if } y = 0 \\ \dots & , \text{ if } y \geq 1 \end{cases}$$

The number of bit operations when using this method is $O(\dots)$.

Continued on next page!

(c) Divide-and-Conquer algorithms

(5 p.)

- Karatsuba's algorithm multiplies two n -bit integers in time $O(\dots\dots\dots)$.

When the algorithm is carried out, it is called recursively $\dots\dots\dots$ times (on the first level only).

- Randomized Quickselect finds the element of rank k of n elements in expected time $O(\dots\dots\dots)$.
- Randomized Quicksort sorts n distinct elements in expected time $O(\dots\dots\dots)$.
- If $\mathcal{A}$ is a comparison-based sorting algorithm, it requires time $\Omega(\dots\dots\dots)$ (worst case) to sort n elements.

Task 2 (Primality Testing)

[8 points]

(a) Formulate the Fermat test:

(2 p.)

Input: Odd integer $N \geq 3$ **Method:**

```

1  choose integer  $a$  ..... from .....;
2  calculate integer  $b \leftarrow$  .....;
3  if ..... then return 1 else return 0;

```

(b) Describe the **output behavior** of the Fermat test:

(2 p.)

- If N is prime, then the output is always
- If N is a composite non-Carmichael number, then $\Pr(\text{output is 0})$
(Possible answers: " $< c$ ", " $\leq c$ ", " $= c$ ", " $\geq c$ ", or " $> c$ " for some appropriate $0 \leq c \leq 1$.)

(c) Let $N \geq 9$ be some composite odd number. An integer $1 \leq a < N$ is called "**bad**" if

(3 p.)

..... and "**good**" otherwise. N is called a **Carmichael number** if

.....

.....

The (iterated) Fermat test does not behave well on Carmichael numbers. To resolve this problem, one can use the test.

(d) The expected running time for generating a random n -bit prime with error probability $\leq 4^{-\ell}$ is (1 p.) $O(\text{.....})$.

Task 3 (RSA)

[6 points]

We consider the RSA cryptosystem as discussed in class. In the following three parts, fill in the missing words or formulas!

(a) Preprocessing/Key generation

(4 p.)

We randomly choose two distinct primes p and q , define $N = p \cdot q$, and calculate $\varphi(N) = \dots$.

For Bob's public key (N, e) , we choose e , $3 \leq e < \varphi(N)$, such that $\dots$.

Bob's private key is (N, d) , where d , $3 \leq d < \varphi(N)$, is such that $\dots$.

d can be calculated using the $\dots$ algorithm.

(b) Communication

(1 p.)

Alice has a message/plaintext $x \in [N] = \{0, 1, \dots, N-1\}$. She computes $y = \dots$ and

sends y to Bob. Bob receives y and computes $z = \dots$.

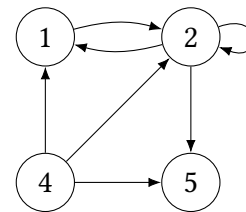
(c) Correctness claim/Decryption condition

(1 p.)

$\dots \bmod \dots = x$ for all $x \in [N]$.

Task 5 (Digraph Representations)

[14 points]

(a) Consider the directed graph G as depicted on the right.Write down G using the following representations:

(6 p.)

(i) Adjacency matrix

	1	2	3	4	5
1					
2					
3					
4					
5					

(ii) Adjacency lists (Use $\boxed{a} \bullet \rightarrow$ for list elements with entry a and non-null pointers, and $\boxed{/}$ for null pointers.)

1	
2	
3	
4	
5	

(iii) Adjacency array

	1	2	3	4	5	6	7
neighbor:							
start:							
	1	2	3	4	5	6	

(b) Draw the **reverse graph** of the digraph depicted in (a).

(2 p.)

(c) Let $G = (V, E)$ be a digraph with $|V| = n$ and $|E| = m$. For a node $v \in V$, $\text{outdeg}(v)$ is the number of outgoing edges $(v, w) \in E$ and $\text{indeg}(v)$ is the number of ingoing edges $(u, v) \in E$. What is the **time** needed to compute these numbers if G is given in one of the following representations?¹

(6 p.)

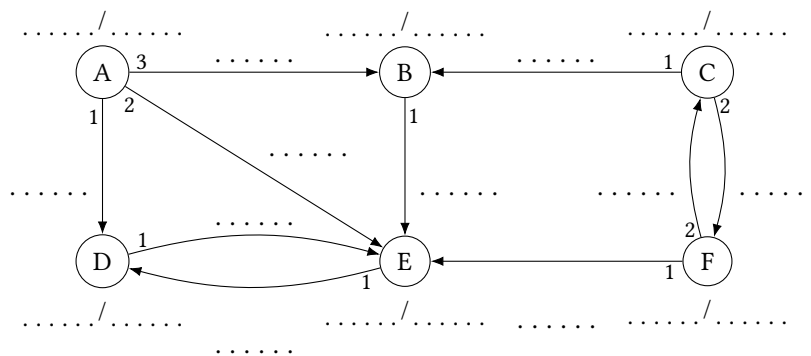
	Adjacency matrix	Adjacency lists	Adjacency array
$\text{outdeg}(v)$:	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$
$\text{indeg}(v)$:	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$	$O(\dots\dots\dots)$

¹In general, $O(n + m)$ is not the same as $O(n)$ or $O(m)$. A bound $O(\text{outdeg}(v))$ can be much smaller than $O(n)$ and $O(m)$.

Task 6 (Depth-First Search – DFS)

[11 points]

- (a) Let $G = (V, E)$ be a digraph with $n = |V|$ nodes and $m = |E|$ edges. **DFS** assigns each node two numbers called, and each edge is labeled with one of the following edge labels: The running time is $O(\dots)$.
- (b) Perform **depth-first search** on the following digraph. The nodes are ordered alphabetically, i. e., start with node A. The numbers on outgoing edges of a node indicate the ordering of its neighbors in its adjacency list. (6 p.)



- (c) A **topological ordering** of G is an ordering $(v_1, \dots, v_n)$ of the nodes such that (2 p.)

.....
 Given the result of a DFS run in G , a topological ordering of G can be computed as follows (if it exists):

.....

Task 7 (Shortest Paths – Dijkstra's Algorithm)

[9 points]

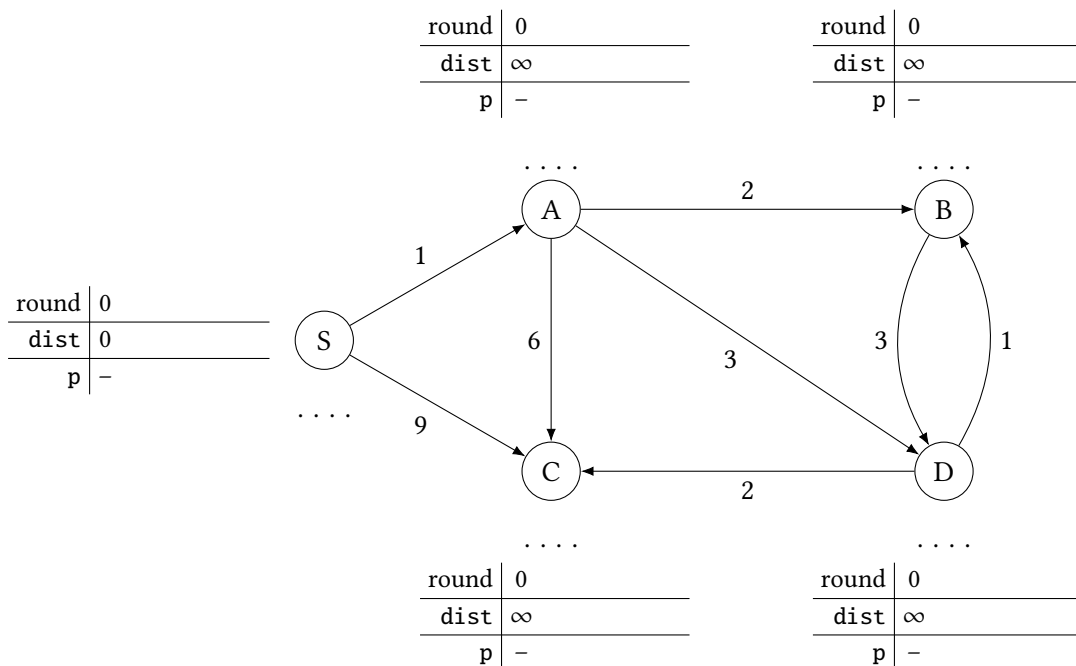
(a) Dijkstra's algorithm can be applied when a weighted digraph $G = (V, E, c)$ and a start node $s \in V$ are given. It calculates the distances $d(s, v)$, for all $v \in V$. All edge weights $c(u, v)$, $(u, v) \in E$, must be The running time of Dijkstra's algorithm (using a priority queue, implemented as a binary heap) is $O(\dots\dots\dots)$, where $n = |V|$ and $m = |E|$. (2 p.)

(b) Run Dijkstra's algorithm on the following weighted digraph, starting at node S, by filling in the gaps and tables as follows: (5 p.)

- If node v is **scanned** in round i , fill in the dotted gap next to v with i .
- If node v is **found** or its distance value $\text{dist}[v]$ is **changed (decreased)** in round i , add a new column to v 's table recording the round number i , the new distance value $\text{dist}[v]$, and the corresponding predecessor $p[v]$.

Ⓢ **13**

round	0	2	5	11
dist	∞	24	12	10
p	-	X	Y	Z



Draw the resulting shortest-path tree.

(2 p.)

Task 8 (Minimum Spanning Trees – Jarník/Prim and Kruskal's Algorithm)

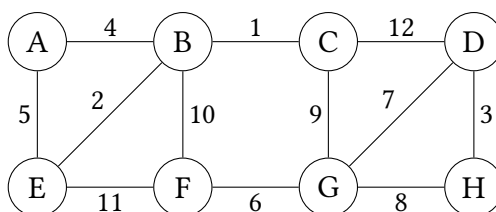
[11 points]

In this task, $G = (V, E, c)$ is a connected (undirected) graph with edge weights $c(e)$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

- (a) A **minimum spanning tree** (MST) for G is an edge set $T \subseteq E$ such that (2 p.)

.....

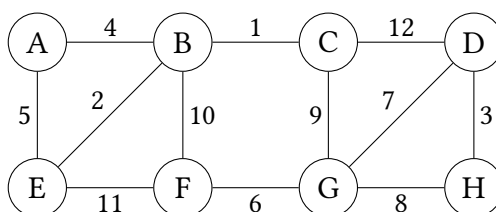
- (b) Run the **Jarník/Prim algorithm** on the following undirected weighted graph with start node A by listing all edges of the resulting minimum spanning tree T in the order in which they are chosen by the algorithm. Calculate the weight of T .



Chosen edges: (A,B), (3 p.)

$c(T) =$ (1 p.)

- (c) Run **Kruskal's algorithm** on the following undirected weighted graph by listing all edges of the resulting minimum spanning tree T in the order in which they are chosen by the algorithm.



Chosen edges: (3 p.)

- (d) The Jarník/Prim algorithm finds a minimum spanning tree in time $O(\dots\dots\dots)$ using a priority queue, realized as a binary heap. (1 p.)

Kruskal's algorithm finds a minimum spanning tree in time $O(\dots\dots\dots)$ using the union-find data structure, realized as trees. (1 p.)

Task 9 (Optimal Binary Coding Trees – Huffman's Algorithm)

[8 points]

- (a) Consider the coding tree T depicted on the right with the following relative frequencies: $p(A) = 0.4, p(M) = 0.3, p(N) = 0.2, p(P) = 0.1$.

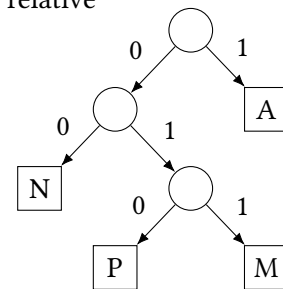
(2 p.)

- (i) Decode 01010010111.

.....

- (ii) Calculate the cost of T .

$$B(T, p) =$$



- (b) The following table gives the frequencies of the letters A, B, ..., G in an unknown language. (The entries have already been sorted according to frequencies.) Run **Huffman's algorithm** on this input by filling in the table. (4 p.)

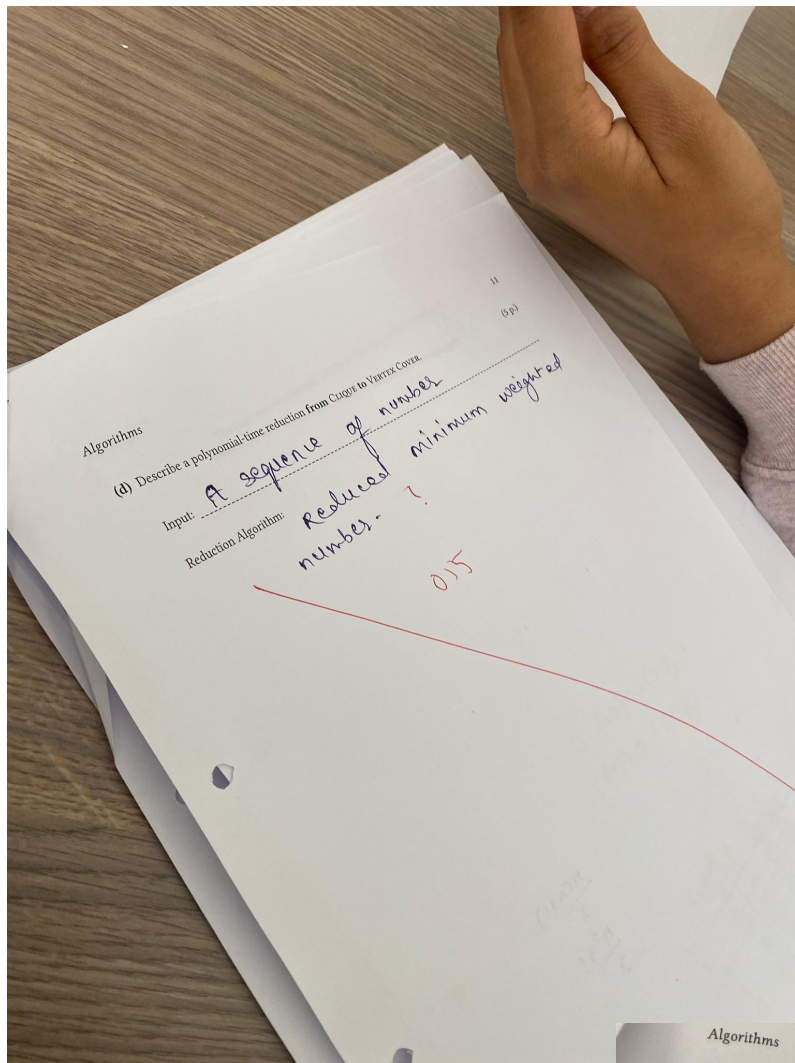
a_i	A	B	C	D	E	F	G						
i	1	2	3	4	5	6	7	8	9	10	11	12	13
p_i	0.04	0.04	0.08	0.10	0.14	0.20	0.40						
pred													–
mark													–

Draw a corresponding optimal coding tree.

(2 p.)

Good luck!





Algorithms

Task 2 (Quicksort)

The QUICKSORT algorithm is called with arguments $a = 1, b = 10$, and with the following global array $A[1..10]$ with entries in $\mathbb{N}$:

5	7	2	1	8	4	9	3	10	6
1	2	3	4	5	6	7	8	9	10

As part of this call, the QUICKSORT ($a = 1, b = 10$) procedure calls (in this order) the subroutines

- PARTITION,
- a first recursive call to QUICKSORT, and
- a second recursive call to QUICKSORT.

Complete the arguments to these subroutines in the following overview and answer the questions:

• QUICKSORT ($a = 1, b = 10$)
The QUICKSORT procedure calls the following subroutines (in this order):

(a) PARTITION ($a = 1 \checkmark, b = 10 \checkmark, p = 5$) (5 p.)
Fill in the state of the global array A directly after this subroutine call has returned:

1	2	5	7	8	4	9	3	10	6
1	2	3	4	5	6	7	8	9	10

The PARTITION procedure call returns the value $r = 1, 2, 5, 7, 8 \checkmark$ (2 p.)

(b) QUICKSORT ($a = 1 \checkmark, b = 10 \checkmark$) (1 p.)
Fill in the state of the global array A directly after this subroutine call has returned:

1	2	3	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10

(c) QUICKSORT ($a = 1 \checkmark, b = 10 \checkmark$) (1 p.)

1	2	3	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10

time complexity $O(n \log n)$

Sorted

Task 4 (Multiplication of Polynomials / FFT)

Let $A(x)$ and $B(x)$ be two polynomials, and let $C(x)$ be the polynomial $C = A \cdot B$. In the lecture you have seen the **Fast Polynomial Multiplication** algorithm, which takes as input $A(x), B(x)$ in coefficient representation, and computes as output the coefficient representation of $C(x)$.

(a) Describe all three phases of the algorithm.

For each phase, give the runtime and name important subroutines.

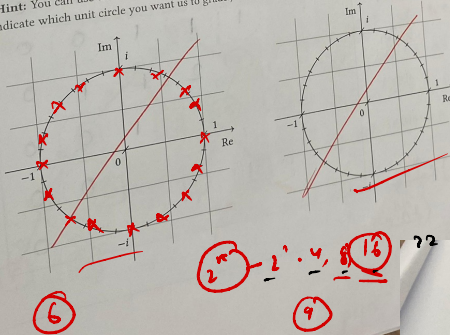
1. using FFT first convert it considers values of $A(n)$ and complete 2 operations. $O(n)$
2. Using $B(x)$ it computes final 2 operation runtime depends on length of input $O(n)$
3. $C(n)$ determined by summation all computation done $A(n) \cdot B(n) = C(n) \leftarrow O(n)$

(b) Let $A(x) = 2x^4 + 12x^2 + 8$ and $B(x) = 7x^5 + 4x^3 + 4x$. What value of n will be used by the algorithm?

$n = 16$

In the unit circle below, mark all n -th complex roots of unity ω^n that will be used by the algorithm during the polynomial multiplication of A and B .

(Hint: You can use the second unit circle if you made any mistakes. If you use both templates, clearly indicate which unit circle you want us to grade.)

**Task 6** (Optimal Binary Coding Trees - Huffman's Algorithm)

(a) Consider the coding tree T depicted on the right with the relative frequencies given below the letter nodes:

(i) Decode 100011010000. REFERENCE

(ii) Calculate the cost of T . State the concrete number, not (only) the formula.

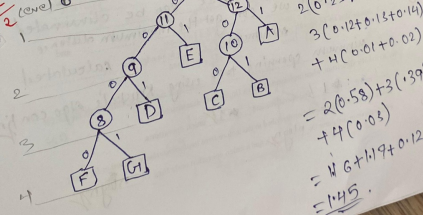
$$\text{cost}(T, p) = 1 \times 0.5 + 0.3 \times 2 + 3 \times 0.1 + 3 \times 0.1 = 0.5 + 0.6 + 0.3 + 0.3 = 1.7$$

(b) The following table gives the frequencies of the letters A, B, ..., G in an unknown language. The entries have already been sorted according to their frequencies.

Run **Huffman's algorithm** on this input by filling in the table. Whenever there is a choice of letters, first pick the one with the smaller index i .

a_i	F	G	D	C	B	E	A
i	1	2	3	4	5	6	7
p_i	0.01	0.02	0.12	0.13	0.14	0.25	0.35
predecessor	8	8	9	10	10	11	12
mark	0	1	1	0	0	1	0

(c) Draw the resulting coding tree T . Draw the child with edge label 0 as the left child.

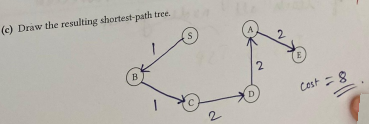
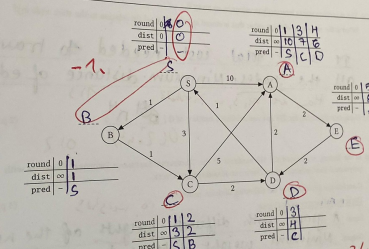


$$\begin{aligned} \text{cost of tree} &= 2(0.25 + 0.35) + 3(0.12 + 0.14 + 0.02) + 4(0.01 + 0.02) \\ &= 2(0.6) + 3(0.28) + 4(0.03) \\ &= 1.2 + 0.84 + 0.12 \\ &= 2.16 \end{aligned}$$

Task 8 (Shortest Paths)

- (a) Let $G = (V, E, c)$ be a weighted Digraph with $n = |V|$ and $m = |E|$.
 The running time of Dijkstra's algorithm (using a binary heap as priority queue) is $O(m \log n + |E| \log n)$.
 For the algorithm to work correctly, the edge weights $c(e)$ must be non-negative for all edges $e \in E$.
 (4 p)

- (b) Run Dijkstra's algorithm on the weighted digraph given below, starting at node S.
 Fill in the gaps and tables as follows (see example on the right).
 • If node X is scanned in round i, write i into the dotted gap next to X.
 • If node X is found or its distance value $\text{dist}[X]$ is changed in round i, add a new column to X's table recording the round number i, the new distance $\text{dist}[X]$, and the corresponding (shortest-path) predecessor $\text{pred}[X]$.
 (5 p)



Task 5 (Longest Increasing Subsequences)

Fill in the recursive solution for computing the values of LISBIGGER. You may abbreviate "LISBIGGER" to "LISB", if you prefer.

$$\text{LISBIGGER}(i, j) = \begin{cases} \text{LISB} & \text{if } L \rightarrow 1 \rightarrow S \rightarrow B \text{ (previous row value) min if } L=i, B=j \\ i \leq j & \text{(1) previous row value or (2) previous row value + 1} \\ \text{minimum} & \text{otherwise} \end{cases}$$

- (b) Execute the Dynamic Programming algorithm LONGESTINCREASINGSUBSEQUENCE to compute the length of a longest increasing subsequence of the following sequence A of $n = 6$ numbers:
 $A[1..n] = (0 \ 4 \ 1 \ 3 \ 7 \ 2)$
 Please use the table below. (The dashed circles mark cells (i, j) that have $A[i] \geq A[j]$)
 (5 p)

	$A[1..n]$	$A[2..n]$	$A[3..n]$	$A[4..n]$	$A[5..n]$	$A[6..n]$
i	1	2	3	4	5	6
$A[0] := -\infty$	5	4	3	2	1	0
$A[1] = 0$		4	3	2	1	0
$A[2] = 4$			1	1	0	0
$A[3] = 1$				2	1	0
$A[4] = 3$					1	0
$A[5] = 7$						0
$A[6] = 2$						
$A[j]$	0	4	1	3	7	2

Task 3 (Problem Descriptions)

Complete the following descriptions of various computational problems by filling in the blanks. (12 points)

(a) SELECTION PROBLEM

Input: A set of numbers or sequence/alphabet. (3 p)

Output: ~~the~~ Desired value through a set of numbers using minimum time. (3 p)

(b) EDIT DISTANCE

Input: Two strings S_1 and S_2 . (3 p)

Output: $S_1 = S_2$ (Insertion, deletion, selection) operations

The edit distance of two strings A, B is defined as the minimum number of operations required to make string 1 equals to string 2 using insertion deletion select. (3 p)

(c) TRAVELING SALESMAN PROBLEM

Input: Graph having nodes eg: A, B, C, D. (3 p)

Output: Shortest distance travelled without repeating any nodes. (3 p)

(d) INDEPENDENT SET (Decision Problem)

Input: a set of sequence. (3 p)

Output: Row and column of increasing sequence by maximum possible sequence. (3 p)

An Independent Set in an undirected graph $G = (V, E)$ is defined as a graph in which $R[i], C[i]$ maximum and it decreased as we traverse further in graph. (3 p)

Task 1 (Mixed Topics)

Fill in the gaps.

(a) Big-O Notation

Let $f: \mathbb{N} \rightarrow \mathbb{R}^+$ and $g: \mathbb{N} \rightarrow \mathbb{R}^+$ be functions.

- We say " $f(n) = \Theta(g(n))$ ", if $g(n) = k \cdot f(n)$ and $n = n_0$. (2 p)
- If $f_1(n) = \sum_{i=1}^n i$, then $f_1(n) = \Theta(n^2)$. (2 p)

(b) Divide-and-Conquer Algorithms

- Mergesort sorts n elements in time $O(n \log n)$ in the worst case. (4 p)
- Karatsuba's algorithm multiplies two n -bit integers in time $O(n^{\log_2(3)})$. (4 p)
- When the algorithm is executed, it calls itself 3 times at the first level of recursion.
- In the lecture, the COUNTTRIANGLES-algorithm (that counts triangles in a graph G) was presented as an application of Fast Matrix Multiplication.
- The COUNTTRIANGLES-algorithm starts by computing nodes using Fast Matrix Multiplication, where w is the node of the graph G . (2 p)

(c) Greedy Algorithms

- "A greedy algorithm incrementally constructs the solution for a problem instance by repeatedly splitting the instance into smaller instances of equal size." (2 p)
- Is this statement true? False
- The greedy EVENT SCHEDULING algorithm repeatedly chooses, from among the selectable events, an event that has minimum time or minimum distance.

Task 9 (TSP and NP-hard Problems)

In the lecture, we saw this Dynamic Programming algorithm that solves the TRAVELING SALESMAN PROBLEM:

Algorithm (Traveling Salesman Problem):

Input: Graph $G = (V, E)$, cost function $c : E \rightarrow \mathbb{R}_0^+$

- 1 Choose start $v_1 \in V$.
- 2 for $v \in V \setminus \{v_1\}$ do
- 3 $P[\emptyset, v] \leftarrow c(v_1, v)$
- 4 for $k = 1, \dots, n-2$ do
- 5 for $S \subseteq V \setminus \{v_1\}$ and $|S| = k$ do
- 6 for $w \notin S \cup \{v_1\}$ do
- 7 $P[S, w] \leftarrow \min_{u \in S} P[S \setminus \{u\}, u] + c(u, w)$
- 8 return $\min_{w \in V \setminus \{v_1\}} P[V \setminus \{v_1, w\}, w] + c(w, v_1)$

(a) The algorithm stores intermediate values in an array P . (3 p.)

What is the meaning of the number that is stored at entry $P[S, w]$ (in relation to the start node v_1)?

It is the total cost needed to travel all the nodes. Minimum distance of edges.
How are v_1, S, w related? v_1 is start node, S is set of nodes, w is next node.

(b) Using this algorithm for TSP, we can achieve a runtime of $O(n^2 \cdot 2^n)$. (2 p.)

(c) In the lecture, we proved the NP-hardness of the TRAVELING SALESMAN DECISION PROBLEM with a polynomial-time reduction from DIRECTED HAMILTONIAN CYCLE. (3 p.)

Complete the description of the following decision problem:

DIRECTED HAMILTONIAN CYCLE

Input:

A graph with different weights of the nodes.

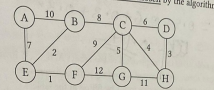
Output:

Minimum weight/distance travelled of each all nodes given.

This is TSP

edge weight function $c : E \rightarrow \mathbb{R}$. You can also assume that G is connected.

- (a) A spanning tree T of G is an edge set $T \subseteq E$ such that T is the vertex set of a tree. The cost of a spanning tree T is defined as $c(T) = \sum_{e \in T} c(e)$. (3 p.)
- (b) Run Kruskal's algorithm on the following undirected weighted graph by listing all edges of the resulting minimum spanning tree T in the order in which they are chosen by the algorithm. (3 p.)



Chosen edges: $(E, F), (E, B), (B, H), (C, H), (C, G), (A, E), (B, C)$

Calculate the weight of T : $\text{weight}(T) = 1+2+3+4+5+7+8 = 30$

(c) A set $R \subseteq E$ is called extendible if $\dots$ minimum edges distance, p calculated. (3 p.)

(d) Complete the formulation of the Cut Property: Assume that

1. $R \subseteq E$ is a part of graph G and $R \cup \{e\}$ can be eliminated.
2. $(S, V-S)$ is a cut such that $e \in E$ and still we can get the minimum distance and e is an edge that is calculated.
3. assume that $e = (v, w)$ with minimum. (3 p.)

Then minimum spanning tree using Kruskal's Algorithm can be found. (3 p.)

(e) Kruskal's algorithm, when implemented with a Union-Find data structure, needs exactly $\dots$ union operations and $\dots$ union operations. In the Union-Find implementation described in the lecture (tree implementation with union by rank) a find operation costs time $O(\log n)$ and a union operation costs time $O(\log n)$.

Exam “Algorithms” SS 2020

September 28, 2020

Exam duration: 90 minutes

Remarks

- (1) Use a blue or black ballpoint pen, or a fountain pen. (Neither pencils nor red pens.)
- (2) Calculators, lecture notes, cheat sheets, and other *material* are *not* allowed. Mobile phones and similar devices like smart watches must be turned off and packed away.
- (3) You can work on the tasks directly on the work sheets. *Additional sheets* will be handed out at request. Write your name and student ID number on these, too.
- (4) Some tasks contain texts with gaps “.....”. Here you are supposed to fill in a formula, one word or several words, or a short sentence (even if there is no explicit instruction).
- (5) Your answers, e. g., bounds in big-O notation, should be meaningful: “An array of size n can be sorted in time $O(n^{42})$ ” is not wrong, but not worth a point.

Task	1	2	3	4	5	6	7	8	9	Total	Grade
Max. score	16	8	6	7	11	11	10	11	10	90	
Achieved score	...	...	...	...	...	...	...	...	...	...	...

Fill in at the beginning of the exam:

 Last name, first name

 Student ID number

 Major
 (probably: RCSE)

In addition, _____ sheets were handed in.

Fill in at post-exam review:

 Date, signature

Task 1 (Mixed Topics)

[16 points]

Fill in the gaps. If not specified in a different way, each correct entry gives 1 point, and each wrong or omitted entry gives 0 points.

(a) Big-O notation

(5 p.)

In this subtask, each correct entry gives 1/2 point. Let $f: \mathbb{N} \rightarrow \mathbb{R}^+$ and $g: \mathbb{N} \rightarrow \mathbb{R}^+$ be functions.

- We say " $g(n) = \Omega(f(n))$ " if there are constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

..... for all

- *Limit rule:* If $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = c$ for some c , then

Domination rule: If $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$, then

- If $f(n) = \frac{(n^5 + 1) \cdot 9^{n/2}}{4n^3 + 5\sqrt{n}}$, then $f(n) = \Theta(n^a \cdot b^n)$ for constants $a = \dots$ and $b = \dots$.
- Let $n \in \mathbb{N}$. Then $\sum_{i=0}^n i = \Theta(\dots)$ and $\sum_{i=0}^n 2^i = \Theta(\dots)$.
- If $f(n) = 2 + \sin(n)$, then $f(n) = O(\dots)$ and $f(n) = \Omega(\dots)$.

(b) Modular arithmetic

(6 p.)

Let $x, y, N \in \mathbb{N}$ be n -bit integers such that $N \geq 2$.

- The value $(x \cdot y) \bmod N$ can be computed naively using grade school methods.

The number of bit operations when using these methods is $O(\dots)$.

- If $x < N$, the value $x^y \bmod N$ can be computed using the following recursive formula:

$$x^y \bmod N = \begin{cases} 1 & , \text{ if } y = 0 \\ x & , \text{ if } y = 1 \\ \dots & , \text{ if } y \geq 2 \text{ even} \\ \dots & , \text{ if } y \geq 2 \text{ odd} \end{cases}$$

The number of bit operations when using this method is $O(\dots)$.

- If $x \geq y$, the value $\gcd(x, y)$ can be computed using the following recursive formula:

$$\gcd(x, y) = \begin{cases} x & , \text{ if } y = 0 \\ \dots & , \text{ if } y \geq 1 \end{cases}$$

The number of bit operations when using this method is $O(\dots)$.

Continued on next page!

(c) Divide-and-Conquer algorithms

(5 p.)

- Karatsuba's algorithm multiplies two n -bit integers in time $O(\dots\dots\dots)$.

When the algorithm is carried out, it is called recursively $\dots\dots\dots$ times (on the first level only).

- Mergesort sorts n elements in time $O(\dots\dots\dots)$ (worst case).
- Randomized Quicksort sorts n distinct elements in expected time $O(\dots\dots\dots)$.
- Randomized Quickselect finds the element of rank k of n elements in expected time $O(\dots\dots\dots)$.

Task 2 (Primality Testing)

[8 points]

(a) Formulate the Fermat test:

(2 p.)

Input: Odd integer $N \geq 3$ **Method:**

```

1  choose integer  $a$  ..... from .....;
2  calculate integer  $b \leftarrow$  .....;
3  if ..... then return 1 else return 0;

```

(b) Describe the **output behavior** of the Fermat test:

(2 p.)

- If N is prime, then the output is always
- If N is a composite non-Carmichael number, then $\Pr(\text{output is } 0)$

(Possible answers: " $< c$ ", " $\leq c$ ", " $= c$ ", " $\geq c$ ", or " $> c$ " for some appropriate $0 \leq c \leq 1$.)(c) Let $N \geq 9$ be some composite odd number. An integer $1 \leq a < N$ is called "**bad**" if

(3 p.)

..... and "**good**" otherwise. N is called a **Carmichael number** if

.....

.....

The (iterated) Fermat test does not behave well on Carmichael numbers. To resolve this problem, one can use the test.

(d) Consider the following procedure:

(1 p.)

Randomly pick an n -bit number N until N is prime.On average this procedure will halt within $O(\dots)$ **rounds**.

Task 3 (RSA)

[6 points]

We consider the RSA cryptosystem as discussed in class. In the following three parts, fill in the missing words or formulas!

(a) Preprocessing/Key generation

(4 p.)

We randomly choose two distinct primes p and q , define $N = p \cdot q$, and calculate $\varphi(N) = \dots\dots\dots$.

For Bob's public key (N, e) , we choose e , $3 \leq e < \varphi(N)$, such that $\dots\dots\dots$.

Bob's private key is (N, d) , where d , $3 \leq d < \varphi(N)$, is such that $\dots\dots\dots$.

d can be calculated using the $\dots\dots\dots$ algorithm.

(b) Communication

(1 p.)

Alice has a message/cleartext $x \in [N] = \{0, 1, \dots, N - 1\}$. She computes $y = \dots\dots\dots$ and

sends y to Bob. Bob receives y and computes $z = \dots\dots\dots$.

(c) Correctness claim/Decryption condition

(1 p.)

$\dots\dots\dots \bmod \dots\dots\dots = x$ for all $x \in [N]$.

Task 4 (Recurrence Relations)

[7 points]

- (a) Consider some algorithm A . It solves instances of size $n = 1$ in constant time. If $n > 1$, it divides the input into 5 sub-instances of size $\lfloor n/2 \rfloor$ in time $O(n^3)$, solves these subproblems recursively, and combines the solutions to obtain a solution of the original input in time $O(n^2 \log n)$.

Let $T(n)$ be the running time of A on inputs of size $n \geq 1$. It fulfills the following recurrence relation: (2 p.)

$$T(n) \leq \begin{cases} \dots\dots\dots, & \text{if } n = 1 \\ \dots\dots\dots, & \text{if } n > 1 \end{cases}$$

- (b) Consider some algorithm B . It solves problems of size $n \geq 1$ in time $T(n)$, where

$$T(n) \leq \begin{cases} 4, & \text{if } n = 1 \\ 8 \cdot T(\frac{n}{2}) + 5 \cdot n^3 - n, & \text{if } n > 1 \end{cases}$$

Solve this recurrence relation using the **Master theorem**. What are the values of the important constants a , b , and d ? Which case of the Master theorem applies? (3 p.)

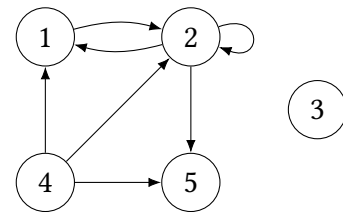
- (c) Consider some algorithm C . On input $n \geq 1$ it calculates $f(n)$, where

$$f(n) = \begin{cases} 1, & \text{if } n = 1 \\ 2 \cdot f(n-1) + 1, & \text{if } n > 1 \end{cases}$$

Solve this recurrence relation **exactly without big-O notation**. (You will need a direct calculation. The Master theorem does not help here.) (2 p.)

Task 5 (Digraph Representations)

[11 points]

(a) Consider the directed graph G as depicted on the right.Write down G using the following representations:

(6 p.)

(i) Adjacency matrix

	1	2	3	4	5
1					
2					
3					
4					
5					

(ii) Adjacency lists (Use $\boxed{a} \bullet \rightarrow$ for list elements with entry a and non-null pointers, and $\boxed{/}$ for null pointers.)

1	
2	
3	
4	
5	

(iii) Adjacency array

	1	2	3	4	5	6	7
neighbor:							
start:							
	1	2	3	4	5	6	

(b) Draw the **reverse graph** of the digraph depicted in (a):

(2 p.)

(c) Let $G = (V, E)$ be a digraph with $|V| = n$ and $|E| = m$. For a node $v \in V$, $\text{outdeg}(v)$ is the number of outgoing edges $(v, w) \in E$. What is the **time** needed to compute $\text{outdeg}(v)$ if G is given in one of the following representations?¹

(3 p.)

Adjacency matrix

Adjacency lists

Adjacency array

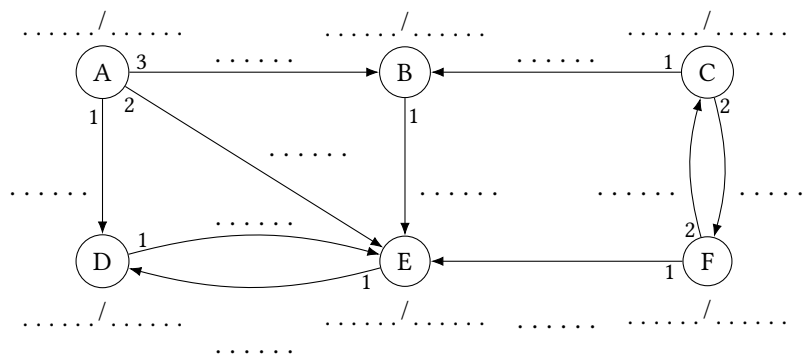
$O(\dots\dots\dots)$ $O(\dots\dots\dots)$ $O(\dots\dots\dots)$

¹In general, $O(n + m)$ is not the same as $O(n)$ or $O(m)$. A bound $O(\text{outdeg}(v))$ can be much smaller than $O(n)$ and $O(m)$.

Task 6 (Depth-First Search – DFS)

[11 points]

- (a) Let $G = (V, E)$ be a digraph with $n = |V|$ nodes and $m = |E|$ edges. **DFS** assigns each node two numbers called, and each edge is labeled with one of the following edge labels: The running time is $O(\dots\dots\dots)$.
- (b) Perform **depth-first search** on the following directed graph. The nodes are ordered alphabetically, i. e., start with node A. The numbers on outgoing edges of a node indicate the ordering of its neighbors in its adjacency list. (6 p.)



- (c) A **topological ordering** of G is an ordering $(v_1, \dots, v_n)$ of the nodes such that (2 p.)

.....
 Given the result of a DFS run in G , a topological ordering of G can be computed as follows (if it exists):

.....

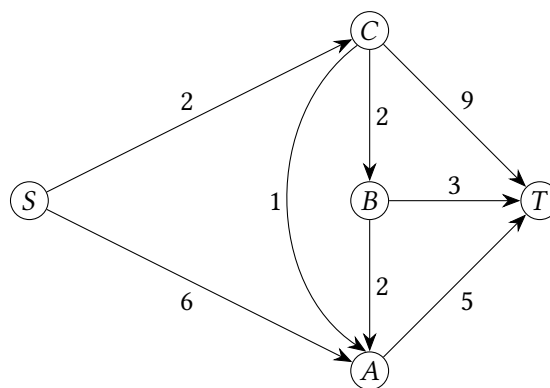
Task 7 (Shortest Paths – Dijkstra's Algorithm)

[10 points]

- (a) Dijkstra's algorithm can be applied when a digraph $G = (V, E)$, a start node $s \in V$, and edge weights $\ell(u, v)$, for $(u, v) \in E$, are given. It calculates the distances $d(s, v)$, for all $v \in V$. All edge weights must be The running time of Dijkstra's algorithm (using binary heaps) is $O(\dots\dots\dots)$. (2 p.)

- (b) Run Dijkstra's algorithm on the following directed graph, starting at node S . Write down the content of the priority queue as a list of triples $(v, \text{dist}[v], \text{prev}[v])$ after each round. Highlight the entry of node v if v will be scanned in the following round. Draw the resulting shortest-path tree.

Hint: In case you are not sure about the priority queue, run the algorithm intuitively.



Content of the priority queue after...

(6 p.)

- initialization: $(S, 0, \text{nil})$
- round 1: $(A, 6, S), (C, 2, S)$
- round 2:
- round 3:
- round 4:
- round 5:

Shortest-path tree:

(2 p.)

Task 8 (Minimum Spanning Trees – Kruskal's Algorithm)

[11 points]

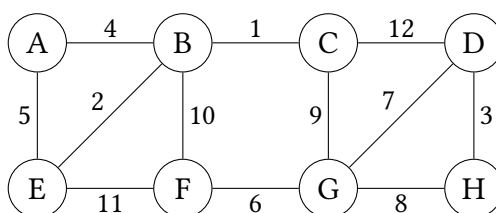
In this task, $G = (V, E)$ is a connected (undirected) graph with edge weights $w(e)$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

(a) A **minimum spanning tree** (MST) in G is an edge set $T \subseteq E$ with the following properties: (2 p.)

(1)

(2)

(b) Run **Kruskal's algorithm** on the following undirected weighted graph by listing all edges of the resulting minimum spanning tree T in the order in which they are chosen by the algorithm. Calculate the weight of T .



Chosen edges: (B, C), (3 p.)

weight(T) = $w(T)$ = (1 p.)

(c) $X \subseteq E$ is called **extendible** if (1 p.)

.....

Formulate the **cut property**: (2 p.)

.....

.....

.....

.....

.....

.....

.....

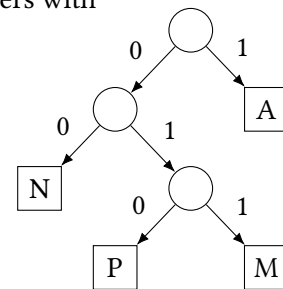
(d) **Kruskal's algorithm** needs exactly find operations and union operations.

A find operation costs time $O(\dots\dots\dots)$, and a union operation costs time $O(\dots\dots\dots)$. (2 p.)

Task 9 (Optimal Binary Coding Trees – Huffman's Algorithm)

[10 points]

- (a) Consider the coding tree T depicted on the right for the following letters with frequencies: (A,40), (M,30), (N,20), (P,10). (2 p.)



- (i) Decode 01010010111.

.....

- (ii) Calculate the cost of T .

cost(T) =

- (b) Assume letters $s_1, s_2, \dots, s_n$ with corresponding frequencies $f_1, f_2, \dots, f_n$ are given. (1 p.)

If s_i and s_j are "two least frequent letters", there is an optimal coding tree in which

.....

- (c) The following table gives the frequencies of the letters A, B, ..., G in an unknown language. (The entries have already been sorted according to frequencies.) Run **Huffman's algorithm** on this input by filling in the table. (4 p.)

s_i	A	B	C	D	E	F	G						
i	1	2	3	4	5	6	7	8	9	10	11	12	13
f_i	2	2	4	5	7	10	20						
P													-

Draw a corresponding optimal coding tree.

(3 p.)

Good luck!

Exam “Algorithms” WS 2019/20

February 24, 2020

Exam duration: 90 minutes

Remarks

- (1) Use a blue or black ballpoint pen, or a fountain pen. (Neither pencils nor red pens.)
- (2) Calculators, lecture notes, cheat sheets, and other *material* are *not* allowed. Mobile phones and similar devices like smart watches must be turned off and packed away.
- (3) You can work on the tasks directly on the work sheets. *Additional sheets* will be handed out at request. Write your name and student ID number on these, too.
- (4) Some tasks contain texts with gaps “.....”. Here you are supposed to fill in a formula, one word or several words, or a short sentence (even if there is no explicit instruction).
- (5) Your answers, e. g., bounds in big-O notation, should be meaningful: “An array of size n can be sorted in time $O(n^{42})$ ” is not wrong, but not worth a point.

Task	1	2	3	4	5	6	7	8	Total	Grade
Max. score	18	7	14	7	12	14	9	9	90	
Achieved score	...	...	...	...	...	...	...	...	...	...

Fill in at the beginning of the exam:

 Last name, first name

 Student ID number

 Major
 (probably: RCSE)

In addition, _____ sheets were handed in.

Fill in at post-exam review:

 Date, signature

Task 1 (Mixed Topics)

[18 points]

Fill in the gaps. If not specified in a different way, each correct entry gives 1 point, and each wrong or omitted entry gives 0 points.

(a) Big-O notation

(4 p.)

In this subtask, each correct entry gives 1/2 point. Let $f: \mathbb{N} \rightarrow \mathbb{R}^+$ and $g: \mathbb{N} \rightarrow \mathbb{R}^+$ be functions.

- We say " $g(n) = O(f(n))$ " if there are $c > 0$ and $n_0 \in \mathbb{N}$ such that

..... for all

- *Limit rule:* If,

then

- If $f(n) = \frac{(n^4 + 1) \cdot 3^{2n}}{n^2 + 5\sqrt{n}}$, then $f(n) = \Theta(n^a \cdot b^n)$ for constants $a = \dots$ and $b = \dots$.

- Let $n \in \mathbb{N}$. Then $\sum_{i=0}^n i = \Theta(\dots)$ and $\sum_{i=0}^n 2^i = \Theta(\dots)$.

(b) Modular arithmetic

(4 p.)

- For $x, y, N \in \mathbb{Z}$, $N \geq 2$, " $x \equiv y \pmod{N}$ " means
- For nonnegative n -bit integers x, y, N , $N \geq 2$, $x < N$, the value $x^y \bmod N$ can be computed using the following recursive formula:

$$x^y \bmod N = \begin{cases} 1 & , \text{ if } y = 0 \\ x & , \text{ if } y = 1 \\ \dots & , \text{ if } y \geq 2 \text{ even} \\ \dots & , \text{ if } y \geq 2 \text{ odd} \end{cases}$$

The number of bit operations when using this method is $O(\dots)$.

(c) Divide-and-Conquer algorithms

(5 p.)

- Karatsuba's algorithm multiplies two n -bit integers in time $O(\dots)$.

When the algorithm is carried out, it is called recursively times (on the first level only).

- Mergesort sorts n elements in time $O(\dots)$ (worst case).
- Randomized Quicksort sorts n distinct elements in time $O(\dots)$ (worst case) and expected time $O(\dots)$.

(d) Dijkstra's algorithm

(5 p.)

- Dijkstra's algorithm computes all distances $d(s, v)$ from a node $s \in V$ in a digraph $G = (V, E)$ with edge weights $\ell(u, v)$, for $(u, v) \in E$. All edge weights must be The running time of Dijkstra's algorithm is $O(\dots)$.
- Dijkstra's algorithm uses a priority queue (PQ). In each round, the PQ operation determines the next node u to be scanned. While examining all outgoing edges $(u, v) \in E$, the PQ operation is executed if a new node v is found. If node v is not new, maybe the PQ operation is executed.

Task 2 (Primality Testing)

[7 points]

(a) Formulate the Fermat test:

(2 p.)

Input: Odd integer $N \geq 3$ **Method:**

```

1  choose  $a$  ..... from .....;
2   $b \leftarrow$  .....;
3  if ..... then return 1 else return 0;

```

(b) Describe the behavior of the Fermat test:

(2 p.)

- If N is prime, then
- If N is a composite non-Carmichael number, then $\Pr(\dots)$

(c) The iterated Fermat test returns 0 if and only if ℓ independent repetitions of the simple Fermat test return 0. If N is a composite non-Carmichael number, its behavior is as follows: (1 p.)

$\Pr(\dots)$

(d) A composite odd number $N \geq 9$ is called a Carmichael number if

(1 p.)

.....

.....

(e) Consider the following procedure:

Randomly pick an n -bit number N until N is prime.

On average this procedure will halt within $O(\dots)$ rounds.

(1 p.)

Task 4 (Recurrence Relations)

[7 points]

- (a) Consider some algorithm A . It solves instances of size $n = 1$ in constant time. If $n > 1$, it divides the input into 5 sub-instances of size $\lfloor n/2 \rfloor$ in time $O(n^2)$, solves these subproblems recursively, and combines the solutions to obtain a solution of the original input in time $O(n \log n)$.

Let $T(n)$ be the running time of A on inputs of size $n \geq 1$. It fulfills the following recurrence relation: (2 p.)

$$T(n) \leq \begin{cases} \dots\dots\dots, & \text{if } n = 1 \\ \dots\dots\dots, & \text{if } n > 1 \end{cases}$$

- (b) Consider some algorithm B . It solves problems of size $n \geq 1$ in time $T(n)$, where

$$T(n) \leq \begin{cases} 1, & \text{if } n = 1 \\ 4 \cdot T(\frac{n}{2}) + n^2, & \text{if } n > 1 \end{cases}$$

Solve this recurrence relation using the Master theorem. What are the values of the important constants a , b , and d ? Which case of the Master theorem applies? (3 p.)

- (c) Consider some algorithm C . It solves problems of size $n \geq 1$ in time $T(n)$, where

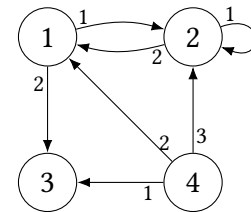
$$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ 2 \cdot T(n-1) + 1, & \text{if } n > 1 \end{cases}$$

Solve this recurrence relation exactly without big-O notation. (You will need a direct calculation. The Master theorem does not help here.) (2 p.)

Task 5 (Digraph Representations)

[12 points]

- (a) Consider the directed graph G as depicted on the right. The numbers on outgoing edges of a node indicate the ordering of its neighbors. Write down G using the following representations:



(i) Adjacency matrix

(2 p.)

	1	2	3	4
1				
2				
3				
4				

(ii) Adjacency lists

(2 p.)

1	
2	
3	
4	

(iii) Adjacency array

(2 p.)

	1	2	3	4	5	6	7
neighbor:							
start:							
	1	2	3	4	5		

- (b) The *reverse graph* of a digraph $G = (V, E)$ is $G^R = (V^R, E^R)$ where

(3 p.)

$V^R = \dots\dots\dots$ and $E^R = \dots\dots\dots$

Draw the reverse graph of the digraph depicted in (a):

- (c) Let $G = (V, E)$ be a digraph with $|V| = n$ and $|E| = m$. For a node $v \in V$, $\text{outdeg}(v)$ is the number of outgoing edges $(v, w) \in E$. What is the time needed to compute $\text{outdeg}(v)$ if G is given in one of the following representations?

(3 p.)

Adjacency matrix

Adjacency lists

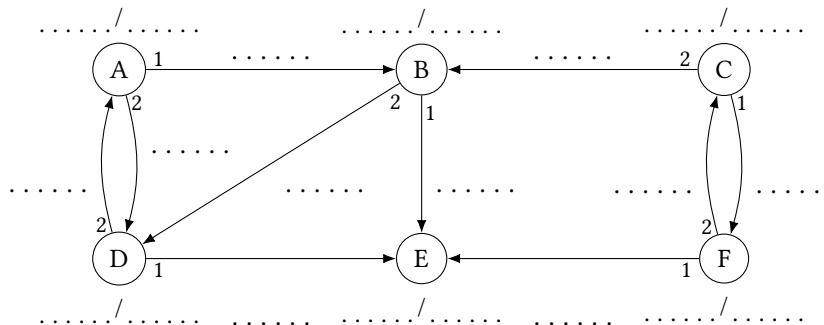
Adjacency array

 $O(\dots\dots\dots)$ $O(\dots\dots\dots)$ $O(\dots\dots\dots)$

Task 6 (Depth-First Search – DFS)

[14 points]

- (a) Perform depth-first search on the following directed graph G . The nodes are ordered alphabetically, i. e., start with node A. The numbers on outgoing edges of a node indicate the ordering of its neighbors in its adjacency list. Classify each edge, and give the pre and post number of each node. (6 p.)



- (b) A *topological ordering* of a digraph $G = (V, E)$ is an ordering $(v_1, \dots, v_n)$ of the nodes such that (1 p.)

.....
 A topological ordering of G exists if and only if G has no cycles. Only considering the result of a DFS run in G , how can we decide if G has a cycle or not? (1 p.)

.....
 If a topological ordering of G exists, one can be computed, given the result of a DFS run in G , as follows: (1 p.)

- (c) What are the *strongly connected components* of the digraph G depicted in (a)? (Give the node sets.) (1 p.)

.....
 How does Kosaraju's algorithm compute strongly connected components of a digraph G ? (3 p.)

(1) Run

(2) Calculate

(3) Run

(4) Output:

What is the running time of Kosaraju's algorithm on input $G = (V, E)$ where $|V| = n$ and $|E| = m$? (1 p.)

.....

Task 7 (Minimum Spanning Trees – Kruskal's Algorithm)

[9 points]

In this task, $G = (V, E)$ is a connected (undirected) graph with edge weights $w(e)$, $e \in E$, $n = |V|$ nodes and $m = |E|$ edges.

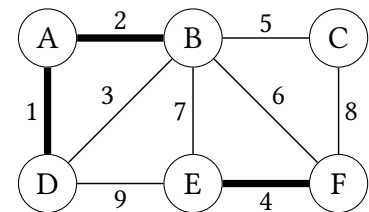
(a) A minimum spanning tree in G is an edge set $T \subseteq E$ with the following properties: (2 p.)

(1)

(2)

(b) We run Kruskal's algorithm implemented with the union-find data structure realized as trees on the graph depicted on the right. The bold edges have already been selected. The last edge we have checked is $e = (E, F)$ with weight 4.

Draw a possible state of the union-find data structure in the current situation, i. e., draw trees corresponding to the sets induced by these edges.



(2 p.)

(c) In the situation of **(b)**, consider the next round: Which edge is checked now? Which operations are executed on the union-find data structure and what are their results (e. g., "find(X) returns Y")? (The results should be consistent with the state of the union-find data structure in your answer for **(b)**.) (3 p.)

.....

.....

.....

.....

.....

(d) Overall, Kruskal's algorithm needs exactly find operations and union operations. A find operation costs time $O(\dots\dots\dots)$, and a union operation costs time $O(\dots\dots\dots)$.

(2 p.)

Task 8 (Optimal Binary Coding Trees – Huffman's Algorithm)

[9 points]

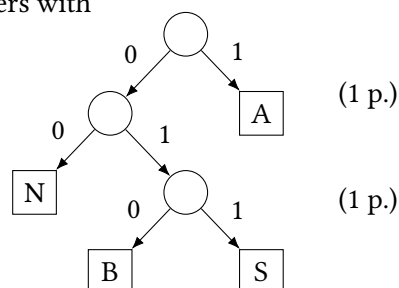
- (a) Consider the coding tree T depicted on the right for the following letters with frequencies: (A,40), (B,10), (N,30), (S,20).

(i) Decode 0101001001011.

.....

- (ii) Calculate the cost of T .

cost(T) =



- (b) The following table gives the frequencies of the letters A, B, ..., G in an unknown language. (The entries have already been sorted according to frequencies.) Run Huffman's algorithm on this input by filling in the table. (4 p.)

s_i	A	B	C	D	E	F	G						
i	1	2	3	4	5	6	7	8	9	10	11	12	13
f_i	1	3	4	6	7	10	19						
p													-

Draw a corresponding optimal binary coding tree.

(3 p.)

Good luck!